

Production Architecture

Session 4. The capstone. Same graph. Nobody at the keyboard.

Saturday 29 August 2026. 13:45 Central.

Rick Hightower. Spillwave. Packt workshop.



35 minutes of teaching. Then 10 minutes to close.

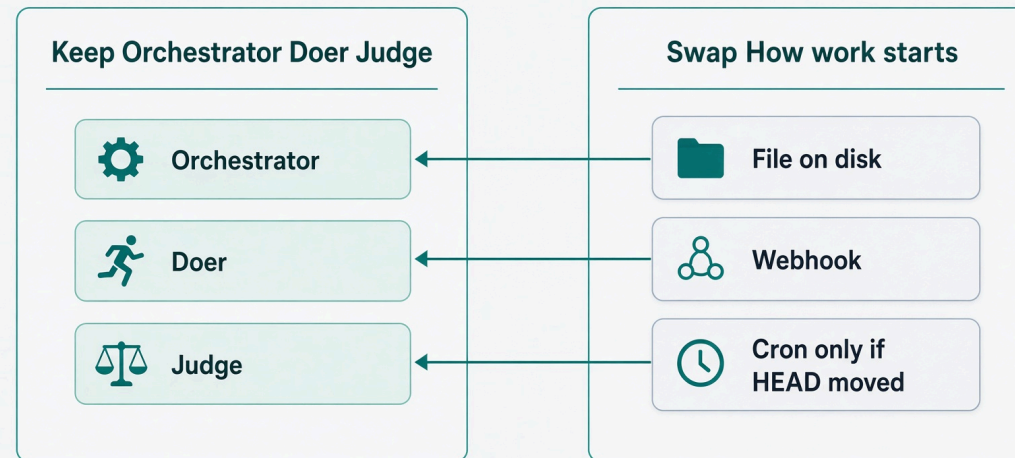
BLOCK	START	MINUTES
State, observability, unattended	13:45	12
Lab 4. Broken pull request fixer	13:57	18
Patterns, adapt to their team	14:15	5
Close. Four artifacts, Monday, questions	14:20	10

The graph does not change. Nobody is at the keyboard.

What changes when you stand up and walk away.

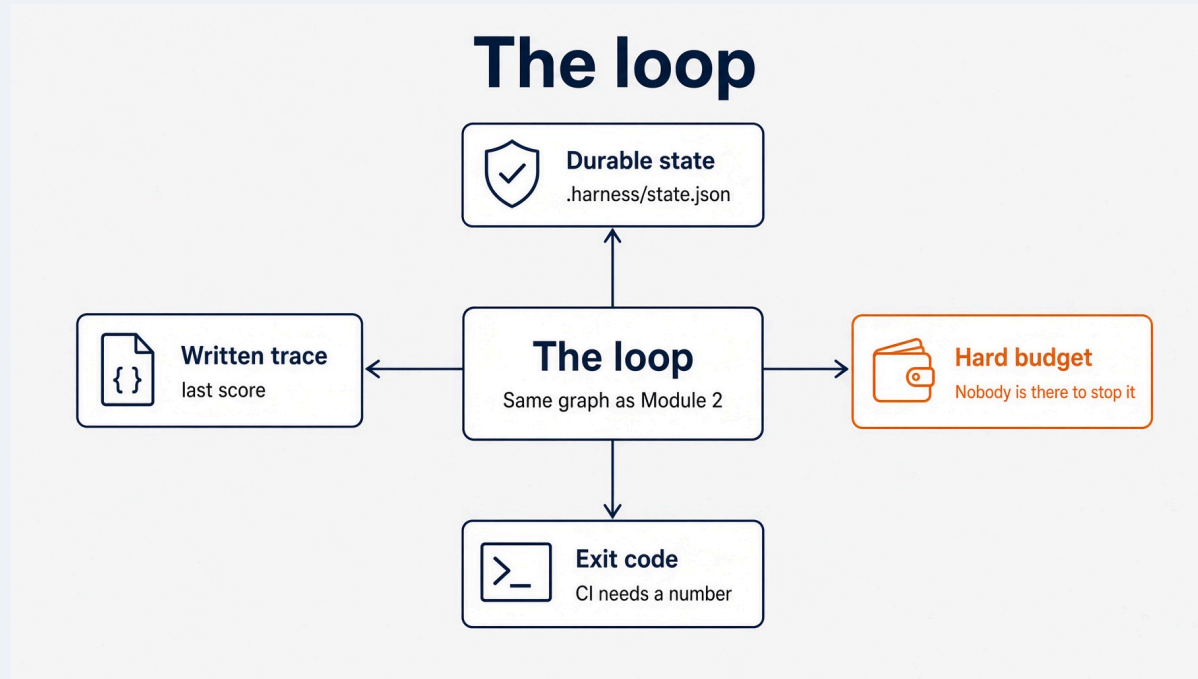
- The graph does not change. The trigger does.
- State has to live on disk. The chat will not be there in the morning.
- The budget stops being advice. Nobody is there to hit Ctrl-C.
- If you cannot read the last score, you cannot debug at 2 a.m.

The graph does not change. The trigger does.



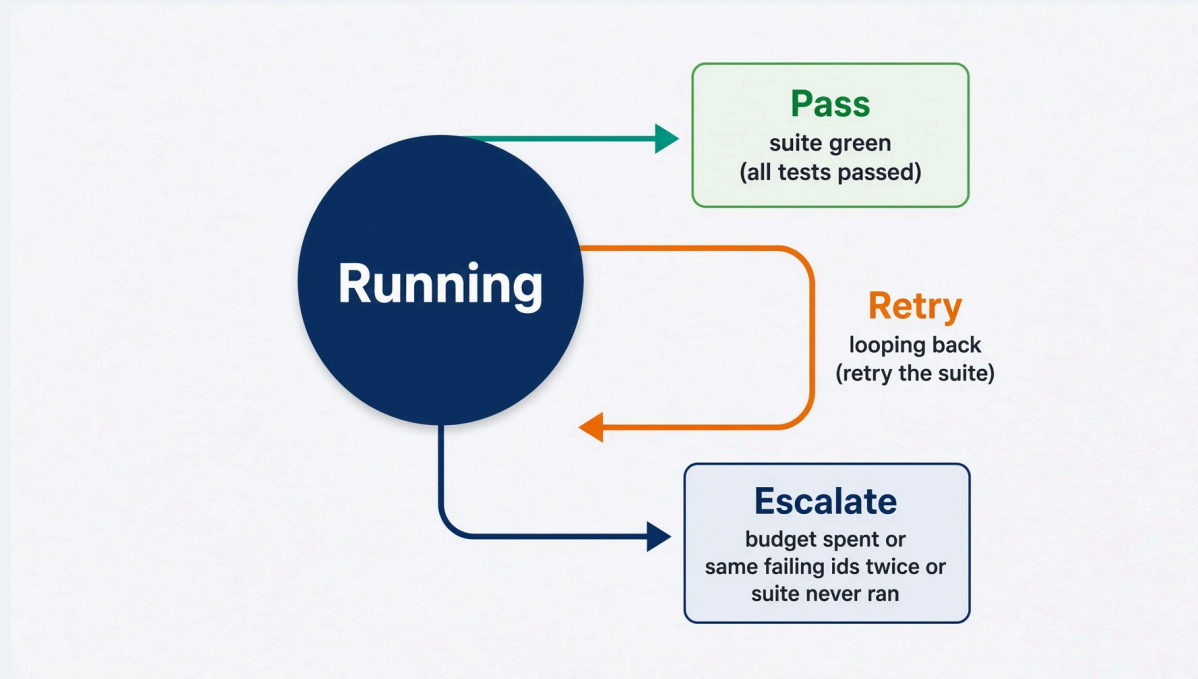
A keystroke is a trigger you cannot ship.

Four things around the loop have to get real.



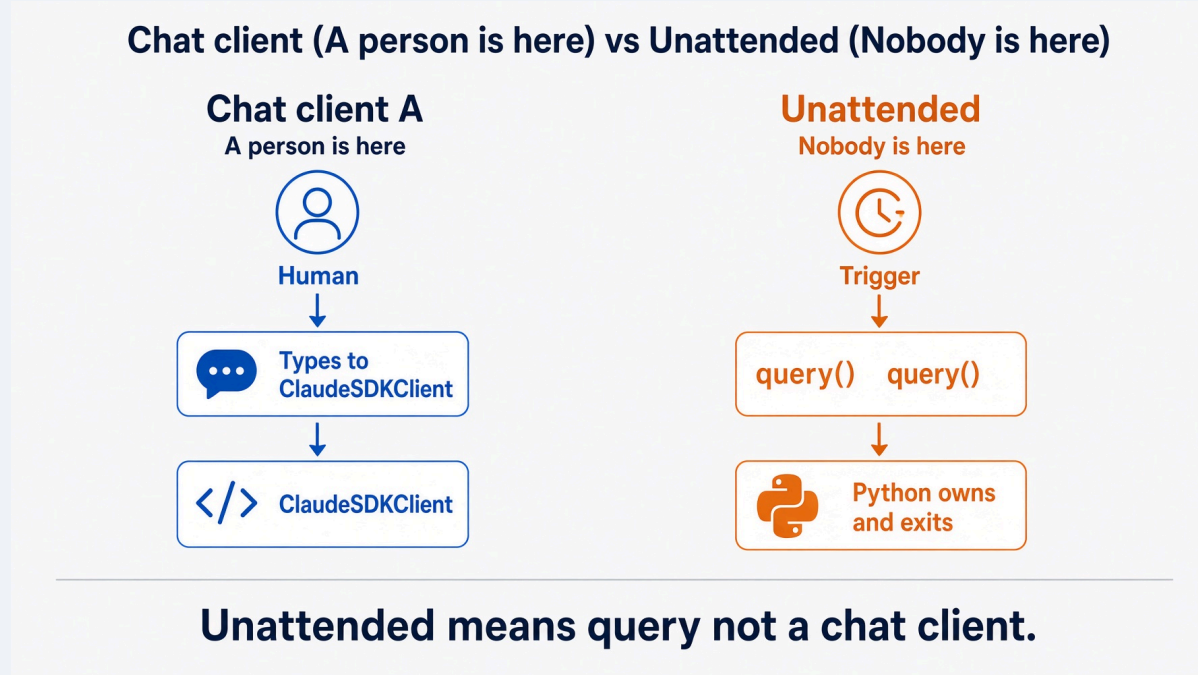
`loops/unattended.py` . The loop does not change. What changes is everything around it.

Python still holds the loop. The model never counts retries.



Pass. Retry. Escalate. No fourth exit.

Unattended means `query()` , not a chat client.

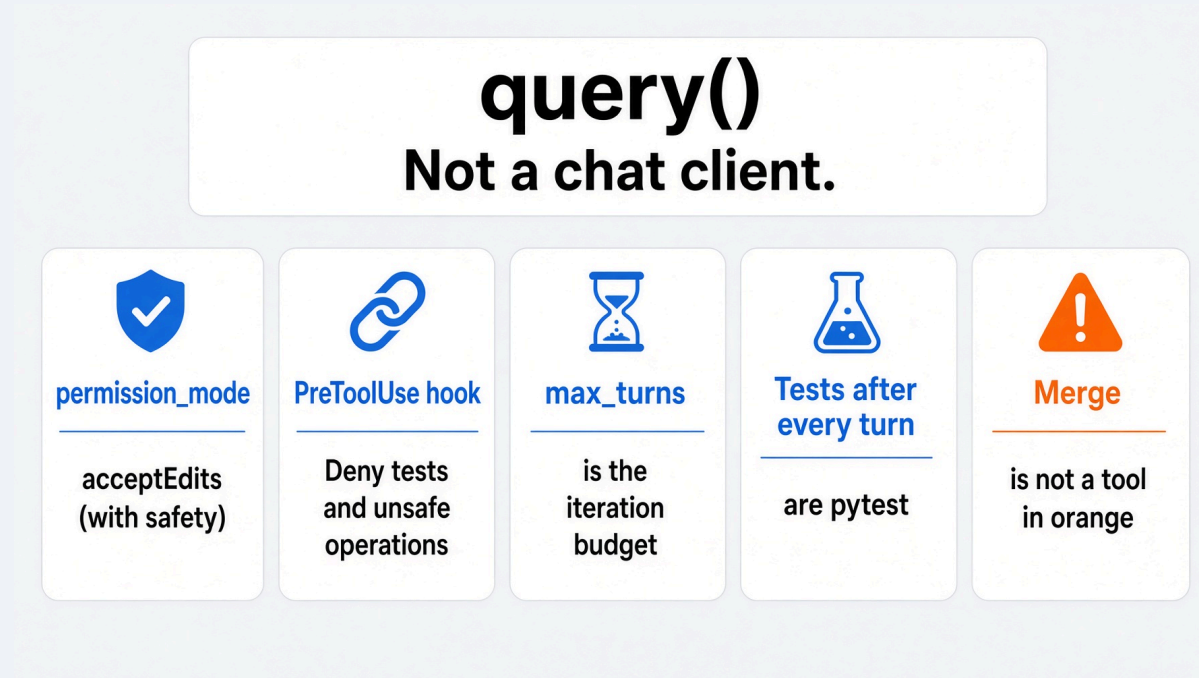


Claude Agent Software Development Kit. One process. A budget. Three exits.

Saturday lab stays two functions in `loop.py` .

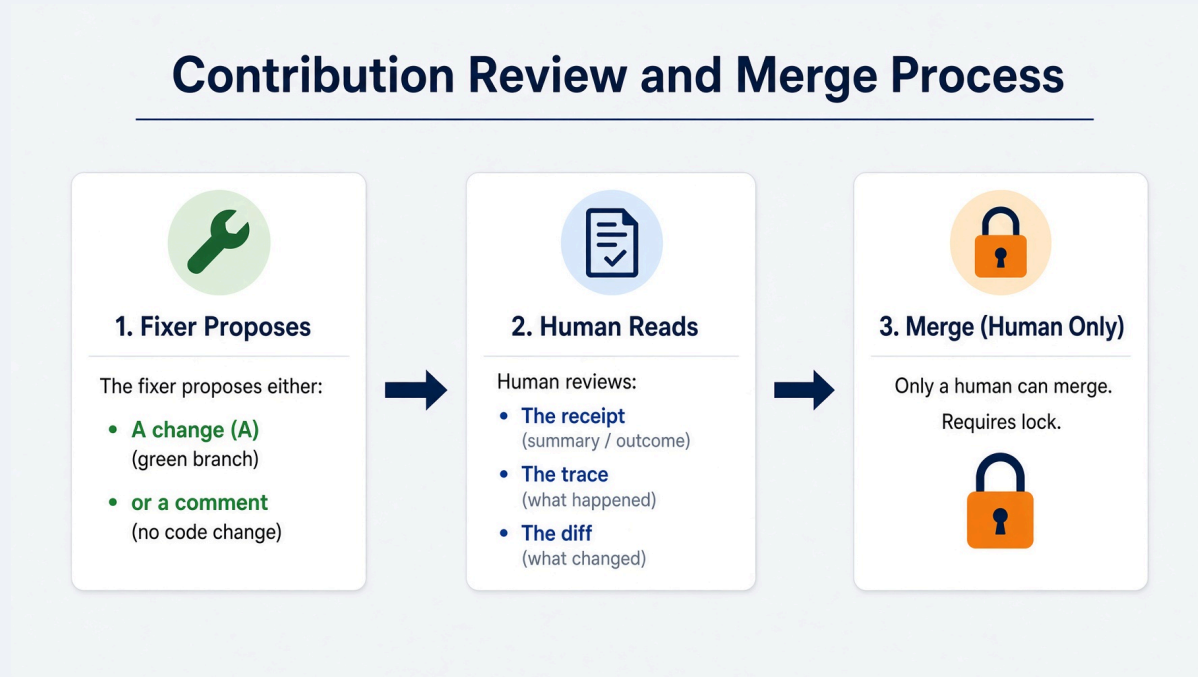
The Agent SDK port is the takehome: `solutions/sol4_fixer_agent_sdk/` . Issue #120.

The Agent SDK contract for a loop with nobody watching.



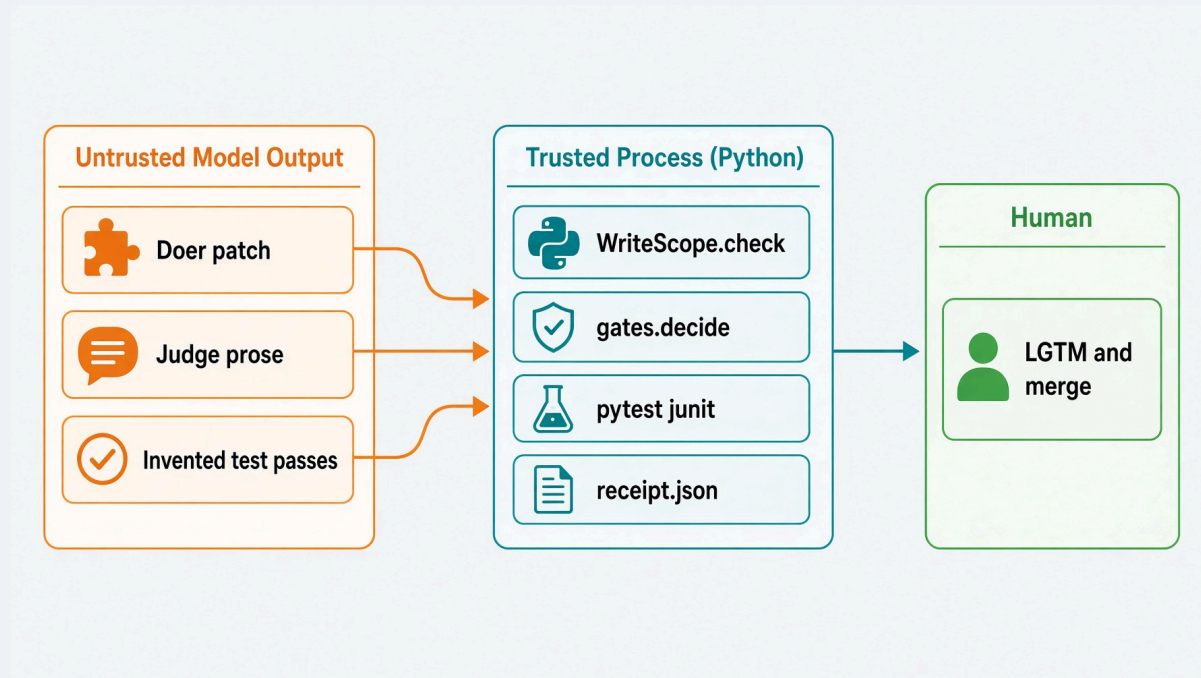
- `permission_mode` : `acceptEdits`
- `PreToolUse`: deny `tests/**`
- `max_turns` is the iteration budget
- Tests after every turn are pytest, not a claim
- Merge stays a human. The loop never gets that tool

Merge stays a human. The loop never gets that tool.



An agent can argue past an instruction. It cannot argue past a tool it was never given.

Trust boundaries do not move because the chair is empty.



Most agent failures are not model failures.

MAST, the Multi-Agent System Failure Taxonomy. 1,642 traces. 7 frameworks.

14 modes, clustered into 3 categories:

CATEGORY	SHARE OF FAILURES
System design issues	41.8%
Inter-agent misalignment	36.9%
Task verification	21.3%

Cemri et al., arXiv:2503.13657, NeurIPS 2025

Every one of those three is something you build, not something you buy.

Fourteen modes. Three categories. This hour is all three.

MAST 14 modes



FC1 is poor specification and missing stops.

FC2 is a breakdown in information flow between agents.

FC3 is a verifier that cannot catch the error, or that stops too soon.

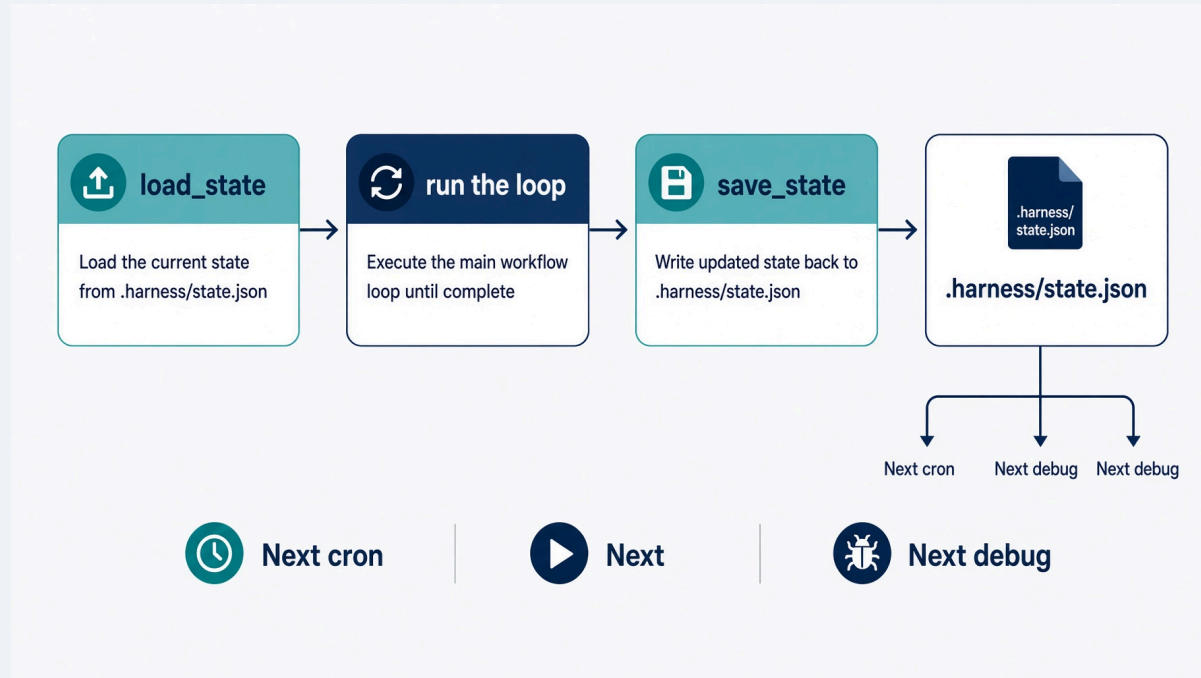
Durable state. Five fields is enough to resume.

```
{  
  "runs": 1,  
  "last_gate": "pass",  
  "last_reason": "the suite is green",  
  "last_run_at": "2026-08-26T15:25:56Z",  
  "loop": "fixer"  
}
```

`.harness/state.json`, in the target repo, next to the receipt.

It survives the process. That is the whole point.

The next run has to know what the last one did.



A corrupt state file is not a fresh start. Say so, then start fresh.

`loops/unattended.py`

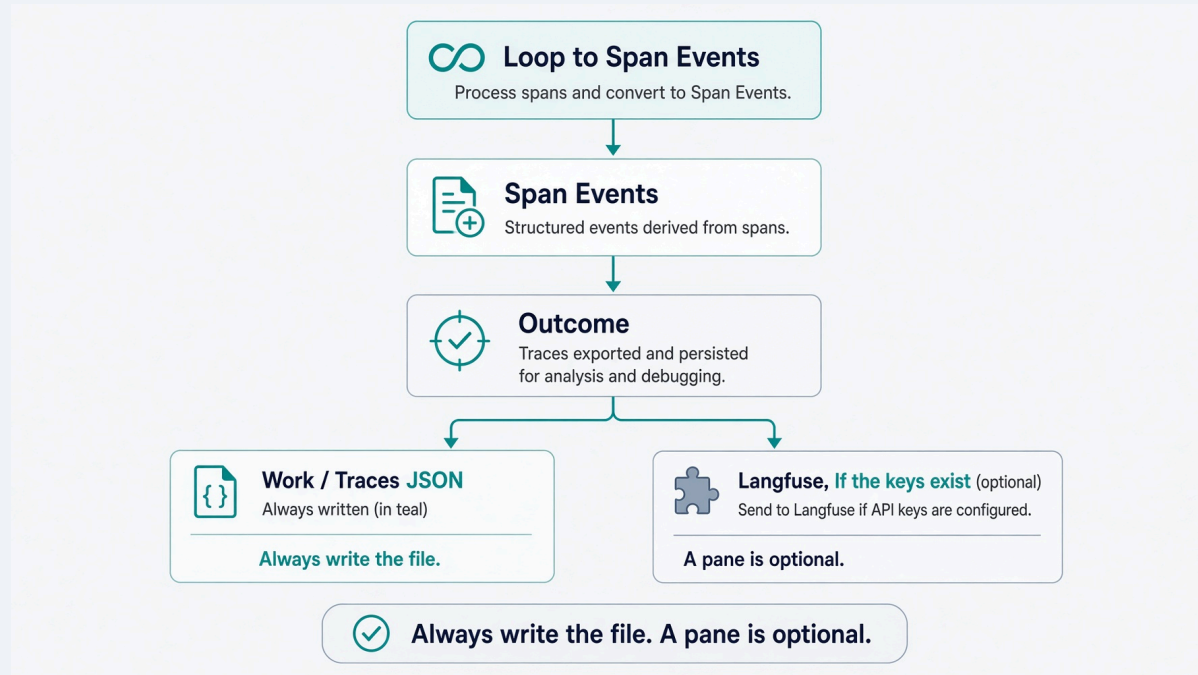
Observability is how you come back.

A trace is not a log file. Each span carries the tool name, the arguments, the output, the duration, the retries, and the error.

Then three numbers per run: steps, loop count, cost per task.

- Local JSON is production, if it is the record you actually open.
- Langfuse is that same record in a pane.
- A dashboard nobody reads is decoration.

Always write the file. A pane is optional.

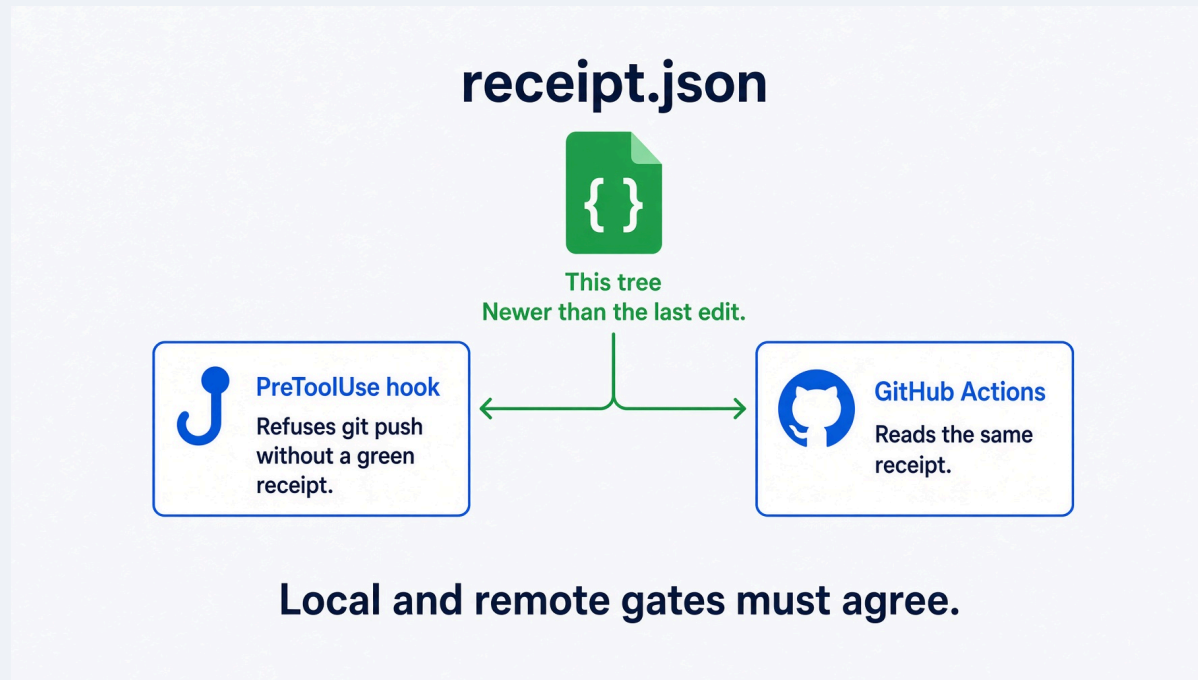


```
with trace("fixer", ticket="broken-pr") as span:
    span.event("red_gate", failing=4)
    span.result(gate="pass", reason="the suite is green")
```

A missing key must never change what the loop does.

solutions/observability.py

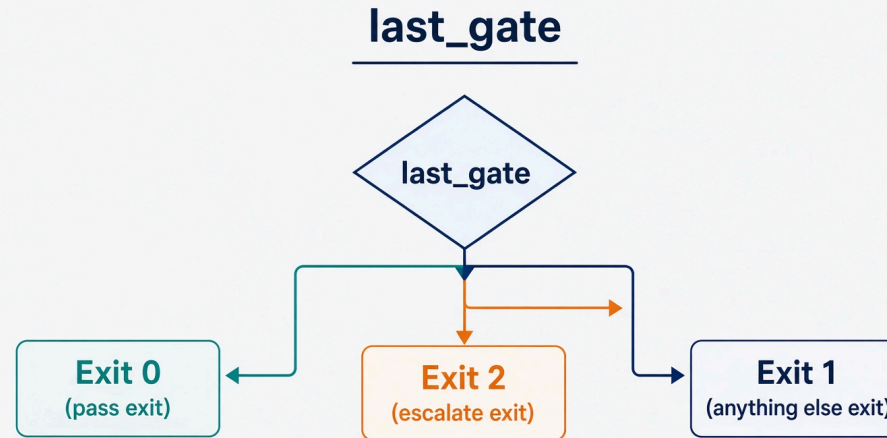
The local gate and the remote gate must agree.



You met the push gate in Module 2. It reads `.harness/receipt.json` and refuses to push without a green one.

Same rule in both places, or the remote one is theater.

Exit 0 is a pass. Exit 2 is an escalation. Exit 1 is a crash.



**Exit 0 is a pass. Exit 2 is an escalation.
Exit 1 is a crash.**

```
# .github/workflows/unattended.yml
on:
  workflow_dispatch: {inputs: {loop: {options: [fixer, implementer, enhancer]}}}
  pull_request:
  schedule: [{cron: "0 15 * * 1-5"}]
```

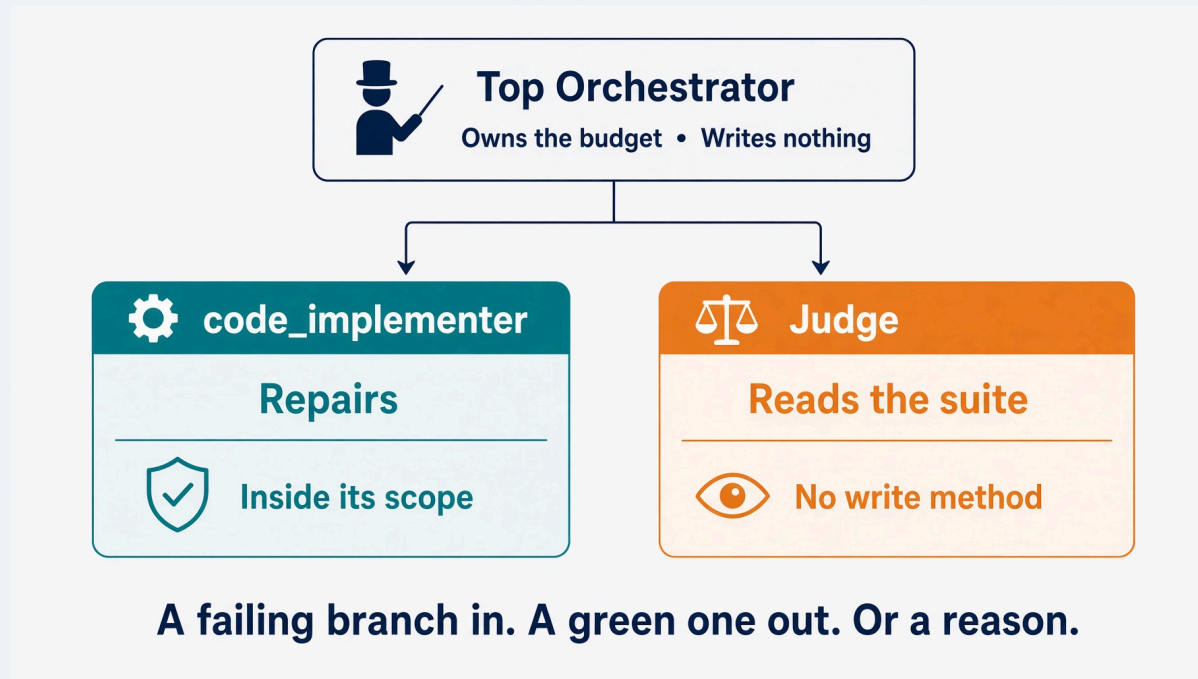
Escalate is not a crash. It is a decision.

You should be at 12 minutes here.

Lab 4. The Broken PR Fixer

18 minutes. A failing branch in. A green one out. Or an honest explanation.

A failing branch in. A green one out. Or a reason.



It is the same graph as the implementer with two differences.

There is no plan to write. The work is already defined by what is red.

Nobody is watching to stop it.

Stash Module 2 first. The loop will not do it for you.

```
git -C ../../work/northwind-field-crm stash --include-untracked
```

loops/fixer.py refuses to clean the tree.

After Module 2 the target repo holds work somebody did.

A loop that quietly deletes it to make its own job easier is the behaviour this workshop exists to prevent.

Fill `loop.py` . Two functions. Nothing else.

Fill `loop.py`

Two functions. Nothing else.

1 summarize_failure



Name the tests



Name the error

2 repair_until_green



then pass



escalate
with a comment

Fill `loop.py`. Two functions. Nothing else.

```
def summarize_failure(run_result: RunResult) -> str:
    raise NotImplementedError("fill me in")

def repair_until_green(contract: Contract, budget: int = 3) -> dict:
    raise NotImplementedError("fill me in")
```

The orchestrator sees the summary, not the whole log.

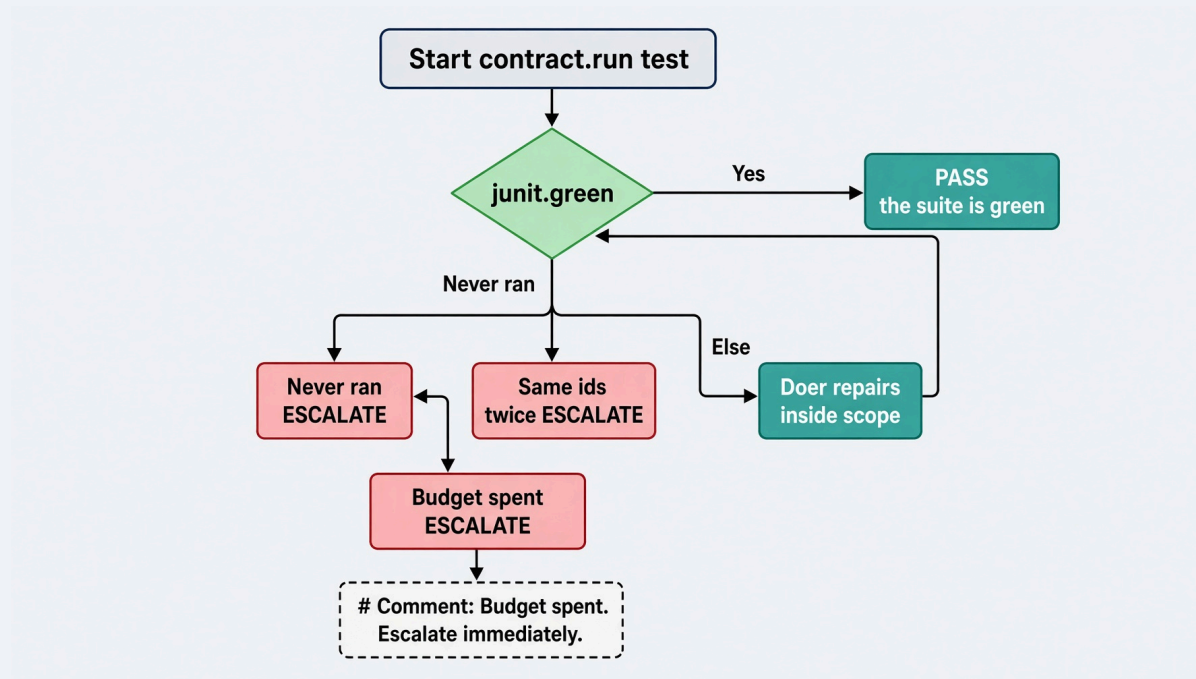
`summarize_failure` . Name the tests. Name the error.

```
def failure_summary(run_result) -> str:
    failed = sorted(run_result.junit.failed_ids)
    lines = [f"{len(failed)} failing: {' '.join(failed[:5])}"]
    error = ERROR_IN_OUTPUT.search(run_result.output or "")
    if error:
        lines.append(error.group(0).strip()[:200])
    return "\n".join(lines)
```

The orchestrator reads this, not the whole log.

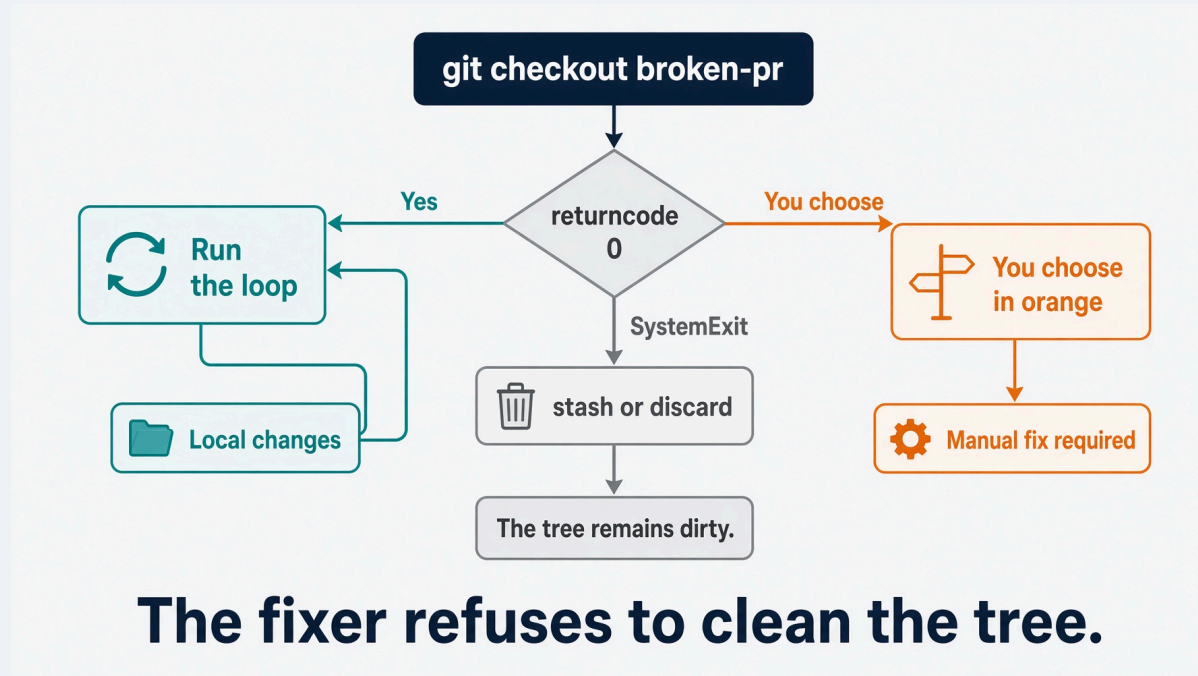
loops/fixer.py

repair_until_green . Stop, and say why.



Giving up is allowed. Giving up silently is the bug.

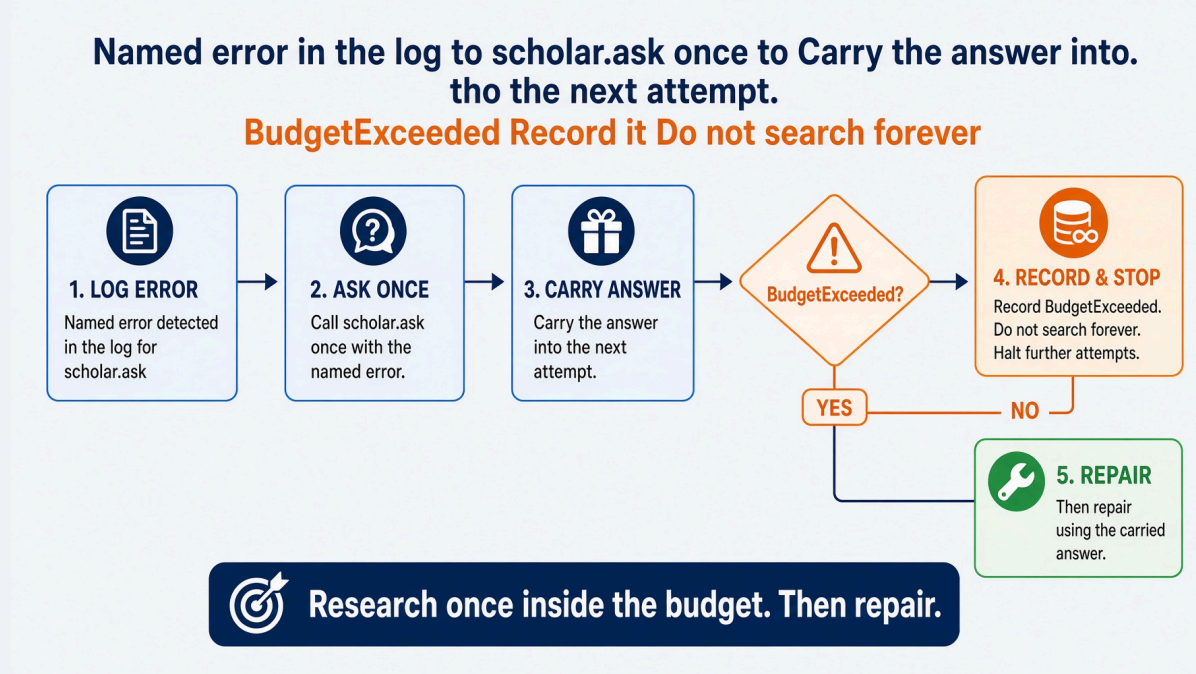
The fixer refuses to clean the tree.



The refusal names both ways out, because an attendee reads it mid-lab with a clock running.

The loop does not decide for the human.

Research once, inside the budget. Then repair.



```
research.Budget(max_calls=2, max_usd=0.05)
```

The attendee decides the backend. The loop cannot tell which one it holds.

Lab 4. The Broken PR Fixer. 18 minutes.

```
cd labs/lab4_fixer
git -C ../../work/northwind-field-crm stash --include-untracked
claude -p "$(cat prompts/claude-code.md)"      # or codex, grok, opencode

task loop:fixer -- --branch broken-pr --doer reference
```

Fill `loop.py` . Two functions: `summarize_failure` and `repair_until_green` .

`--branch broken-pr` is what makes this real.

Falling behind is fine: watch Rick finish `loop.py` and keep going.

Three exits. The comment is the product of the third.

```
attempt 1: 1 failing -> retry
  1 failing: tests.test_overdue::test_overdue_ignores_tasks_with_no_due_date
attempt 2: 1 failing -> escalate

gate: escalate
reason: the same rows failed twice.

The fixer gave up.

Reason: the same rows failed twice: ...
Still failing: [...]
A human should take this one.
```

An iteration that burns tokens and reproduces the identical failure is not progress.

Falling behind is fine. Watch Rick finish.

```
cp loop.py loop.py.my-attempt
```

There is no drop-in `loop.py`. Watch Rick finish and type what he typed.

You continue with a working artifact.

Read `labs/lab4_fixer/FALL-BEHIND.md`.

Nobody leaves this room behind.

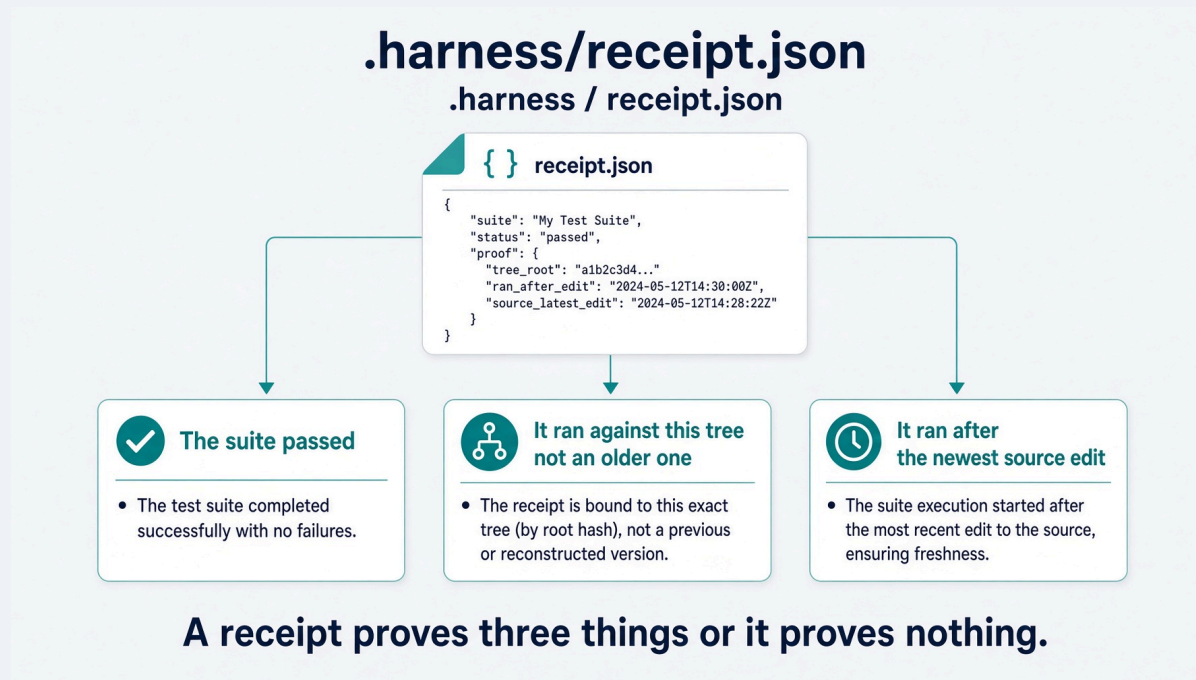
Why the receipt exists at all.

When a model may both act and verify its own work, it can produce **plausible false evidence**: invented test passes, file edits that never happened, fabricated API responses.

That is not hallucination about the world. It is a wrong judgment about the state of its own output.

A self-check cannot catch this by construction.

The receipt is the reason you can trust the run.



A receipt proves three things or it proves nothing.

Python writes it. The model does not.

```
scripts/receipt.py
```

One return per way a receipt can fail to prove its case.

CHECK	WHAT THE ROOM READS
Missing file	No receipt. The suite has not run against this tree.
Unreadable	Treating that as a failure, not a pass.
No junit	No evidence is not the same as clean.
Not green	Last run: FAILED.
Tree changed	The receipt is stale.
Source newer	Re-run the tests.

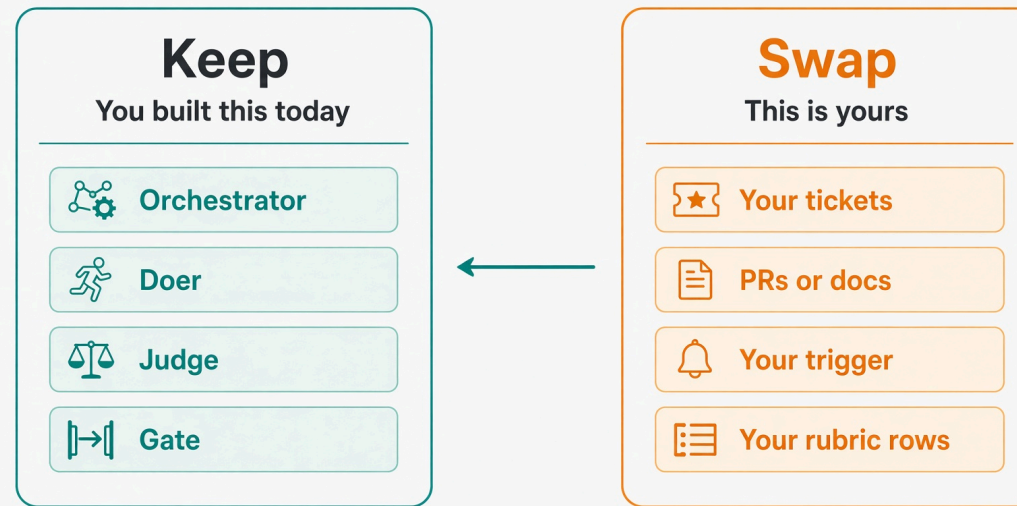
The push gate reads only this file.

You should be at 30 minutes here.

Production loop patterns

Swap the object. Keep the graph. Map home.

Swap the object. Keep the graph.



Four modules, one graph, four objects. That was on purpose.

Same graph. Four objects. You already ran all four.

Same graph: Orchestrator + Doer + Judge



M1 Draft ticket



M2 Ready ticket + code



M3 Question



M4 Failing pull request

You already ran all four.

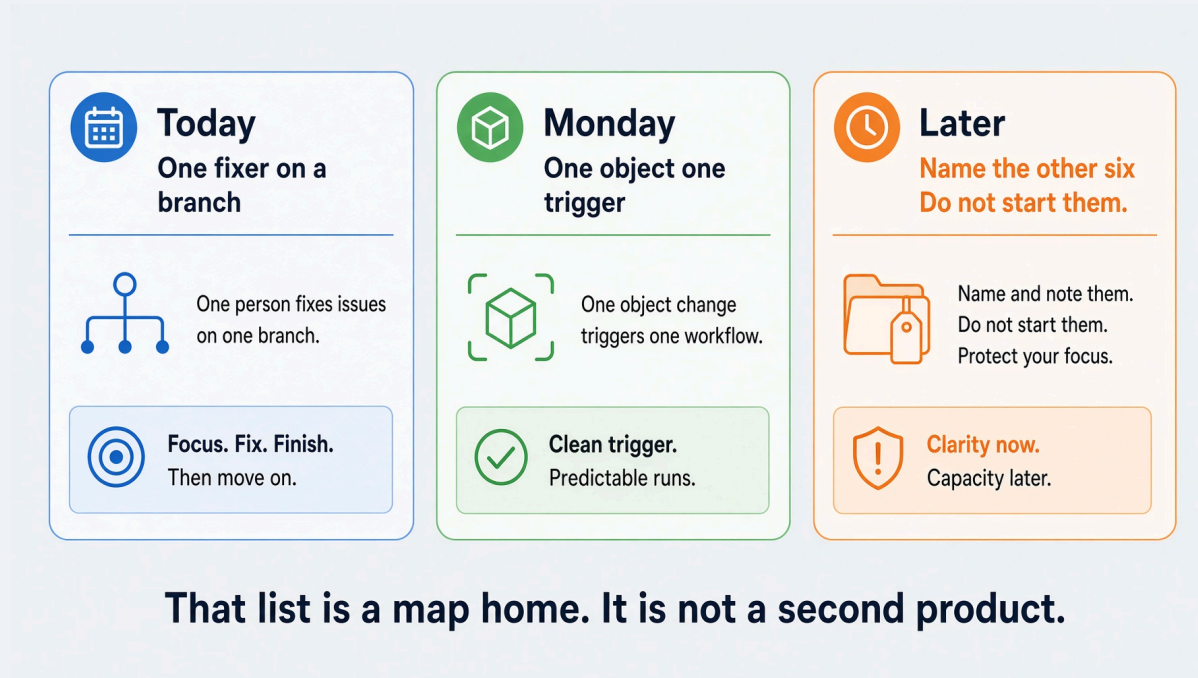
MODULE	OBJECT	LAB
1	A draft ticket	Enhancer
2	A ready ticket, and the code that satisfies it	Implementer
3	A question	Research
4	A failing pull request	Fixer

Seven production loops. Named, not built.

Daily triage. Pull request babysitter. Continuous Integration sweeper. And four more.

LOOP	TRIGGER	GRAPH YOU ALREADY HAVE
Daily triage	cron	enhancer
Ticket groomer	issue opened	enhancer
Pull request babysitter	check suite failure	fixer
Continuous Integration sweeper	cron, if HEAD moved	fixer
Ready-ticket implementer	label ready	implementer
Research brief	a question queue	research
Nightly eval	weekly cron	the harness

That list is a map home. It is not a second product.

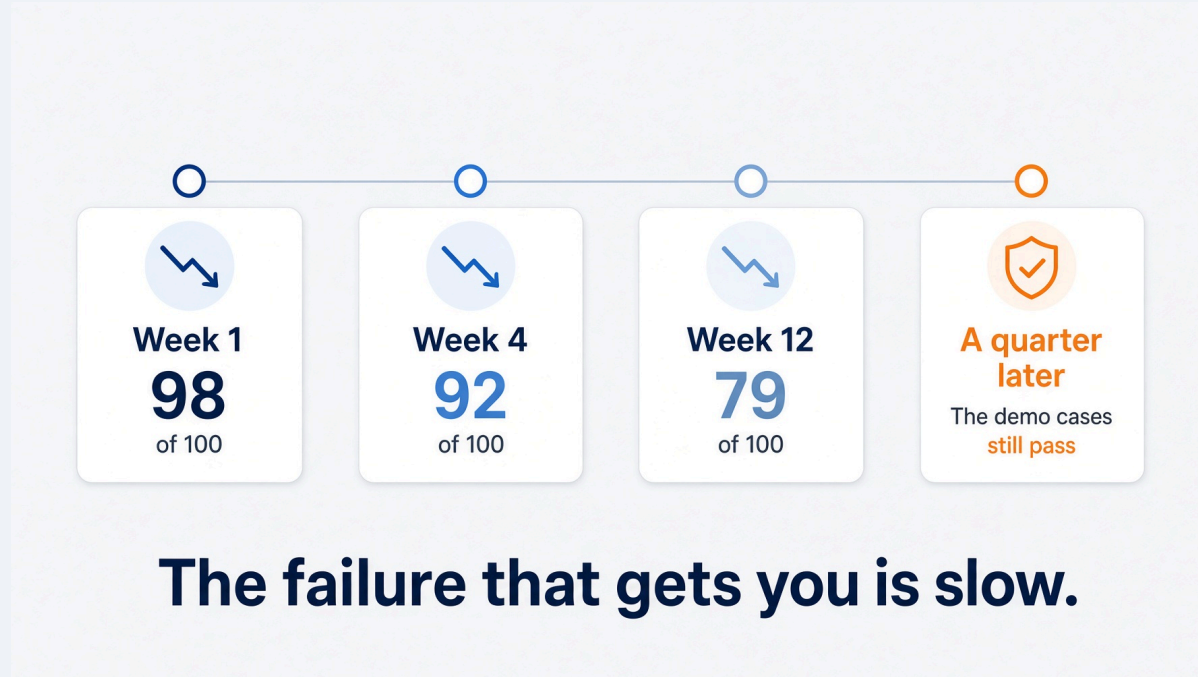


The trigger moves out of the loop. The exits stay in it.

A workflow file starts the run. It never decides when to stop.

Extra credit is `labs/extra-credit/`. Not on the Saturday clock.

The failure that gets you is slow.



A system passes every demo case, earns trust, then degrades over months with no single thing breaking.

The causes are state, context, retrieval, latency, and observability. Not model capability.

So run the evaluation weekly, not quarterly. A 2% weekly drop is invisible in any one week and catastrophic across a quarter.

You should be at 35 minutes here.

Close. 10 minutes.

Four artifacts, where they live, and what you do on Monday.

What you take home.

AI System Development & Deployment Flow

From research loop to production architecture



1. A running autonomous loop. Module 1.
2. A reusable evaluation harness. Module 2.
3. One research assistant over Model Context Protocol. Module 3.
4. A production architecture. Module 4.

All four run on your machine, from a clean clone, with one task setup .

Where everything lives.

labs/	four labs. cd into one and work there.
solutions/	the answer. One standalone folder per lab and runtime.
labs/takehome/	Agent SDK and Deep Agents. Not Saturday.
work/	the target repo, cloned by task setup.

Lab 1 still has one folder per tool. Labs 2 to 4 keep the two runtime ports.

What to do on Monday.

1. Pick one backlog object. Not five.
2. Write one ticket whose criteria a test can fail.
3. Give the target repo a `Taskfile.yml` that emits `junit.xml` .
4. Split the doer before you add a single tool.
5. Arm the push gate on day one, while the loop is still small.

The order is the advice.



Do not add a tool until the doer is split.

Do not skip the gate because the loop is still a prototype.

The refusals are cheap while the loop is small. They are expensive after it has fans.

Takehome. Agent SDK unattended fixer. Issue #120.

```
solutions/sol4_fixer_agent_sdk/
```

- `query()` , **not** `ClaudeSDKClient` . Nobody is chatting.
- `permission_mode` **is** `acceptEdits` .
- `PreToolUse` is write scope. `tests/**` is denied.
- Merge is never a tool.
- Python still owns `summarize_failure` and the three exits.

```
cd solutions/sol4_fixer_agent_sdk
python3 loop.py --table-only
python3 -m pytest tests -q
```

The five-hour labs need no model key. This one does.

Primary references for this session.

- Cemri et al. Why Do Multi-Agent LLM Systems Fail? arXiv:2503.13657. NeurIPS 2025.
- Yao et al. ReAct. arXiv:2210.03629
- `loops/fixer.py` , `loops/unattended.py` , `loops/gates.py`
- `scripts/receipt.py` , `solutions/observability.py`
- `solutions/sol4_fixer_agent_sdk/` , Issue #120
- `labs/lab4_fixer/ARCHITECTURE.md`

Questions.

The loop is the product. The prompt is not.

Six lines to keep. Plus one.

1. A loop is a state machine you enforce.
2. Verify is what separates a loop from a generator.
3. Write scope is a missing method, not a polite request.
4. Three exits. The forgotten one is stable failure.
5. Memory lives on disk. The window is not it.
6. The human still owns irreversible action.

And from this hour: if you cannot read the last score, you cannot debug at 2 a.m.

The loop is the product. The prompt is not.